
boolean



# Node + MySQL

Come comunicano Express ed SQL?

# Hello mysql2!

**mysql2 è un client che permette di far comunicare Node.js e MySQL.**

Fornisce delle funzioni già pronte che si occupano di tutti i dettagli di basso livello e permettono allo sviluppatore Node.js  di avere un'interazione rapida con MySQL. Naturalmente funziona con Express!

*Installiamo il pacchetto*

```
pnpm add mysql2
```

*Creiamo poi un file dedicato*




*All'interno del file, impostiamo le configurazioni di MySQL*

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'root',
  database: 'pizzeria'
});

connection.connect((err) => {
  if (err) throw err;
  console.log('Connected to MySQL!');
});

module.exports = connection;
```

Naturalmente dobbiamo stare attenti ad inserire le giuste informazioni nella funzione **createConnection.**

Se tutto è impostato correttamente, vedremo il console.log e la connessione è riuscita!



# Query al database

Adesso saremo in grado di effettuare query al DB e raccogliere i dati proprio da lì.

Importiamo il file appena creato nel nostro Controller. Poi proviamo a cercare l'elenco delle pizze dal DB:

1) Prepariamo la nostra **query SQL**

2) Usiamo il metodo **query()** passando la query SQL preparata prima e una **callback function.**

```
1
2  // Importiamo il file di connessione al database
3  const connection = require('../data/db');
4
5  // Index
6  router.get('/', (req, res) => {
7    // prepariamo la query
8    const sql = 'SELECT * FROM pizzas';
9
10   // eseguiamo la query!
11   connection.query(sql, (err, results) => {
12     if (err) return res.status(500).json({ error: 'Database query failed' });
13     res.json(results);
14   });
```

La callback function ci fornisce l'eventuale errore e i risultati della query in formato JavaScript!

Non ci resta che convertirli in JSON per restituire la nostra lista!





# Prepared Statements

Una piccola grande accortezza... per la sicurezza!

# SQL Injections

## Hello computer security

Quando per fare delle query abbiamo bisogno di dati provenienti dall'utente, dobbiamo sempre tenere in conto che potrebbero esserci degli utenti mal intenzionati.

Questi utenti potrebbero iniettare del codice SQL che altera o bypassa le nostre query: questa operazione è detta **SQL Injection.**



Pertanto, **prima di arrivare al database**, l'input dell'utente **deve essere sempre:**

- validato
- controllato
- sanitizzato
- “escaped”



*Inserire direttamente l'input dell'utente nella query, come mostrato qui sotto è molto rischioso!*

```
1 // Show
2 router.get('/:id', (req, res) => {
3   // recuperiamo l'id dall' URL
4   const id = req.params.id
5
6   const sql = `SELECT * FROM pizzas WHERE id = ${id}`;
7   //...
8 });
```

# Come evitare le SQL Injections?

## Prepared statements!

Le prepared statements sono **un diverso modo di eseguire le query**: consistono nel "preparare" l'istruzione utilizzando dei **segnaposto** per i valori da inserire nella query.
Questi vengono sostituiti successivamente coi i valori effettivi da inserire nella query, ma non prima di essere stati controllati e "sanificati" dal nostro pacchetto mysql2!

```
1  // Show
2  router.get('/:id', (req, res) => {
3    // recuperiamo l'id dall' URL
4    const id = req.params.id
5
6    const sql = 'SELECT * FROM pizzas WHERE id = ?';
7    connection.query(sql, [id], (err, results) => {
8      if (err) return res.status(500).json({ error: 'Database query failed' });
9      if (results.length === 0) return res.status(404).json({ error: 'Pizza not found' });
10     res.json(results[0]);
11   });
12 });
```

Il secondo parametro della funzione **query()** diventa quindi un array, contenente i valori dei placeholder da controllare.



# Come evitare le SQL Injections?

## Prepared statements!

Se abbiamo diversi parametri, conta l'ordine in cui i nostri **?** sono stati inseriti nella query:

```
1  // recuperiamo l'id dall' URL
2   const { id } = req.params;
3
4   // recuperiamo i dati dal body della richiesta
5   const { name, image } = req.body;
6
7   // prepariamo la query SQL
8   const sql = 'UPDATE pizzas SET name = ?, image = ? WHERE id = ?';
9
10  // Prepariamo la query per aggiornare la pizza
11  connection.query(sql, [name, image, id], (err, results) => {
12      //....
13    }
14  );
```

# Come evitare le SQL Injections?

## Prepared statements!



Dato che i **parametri vengono passati in un secondo momento**, utilizzando un diverso protocollo, le SQL Injections sono inefficaci e non è nemmeno necessario fare l'escaping dei caratteri pericolosi.



# LIVE CODING

# CRUD col DB: Destroy

Ora che abbiamo imparato a connetterci dal DB, proviamo ad effettuare una vera e propria eliminazione!

Nel nostro controller, proviamo ad eseguire una query di cancellazione nella funzione di destroy:

```
1  // Destroy
2  router.delete('/:id', (req, res) => {
3
4    // recuperiamo l'id dall' URL
5    const { id } = req.params;
6
7    //Eliminiamo la pizza dal menu
8    connection.query('DELETE FROM pizzas WHERE id = ?', [id], (err) => {
9      if (err) return res.status(500).json({ error: 'Failed to delete pizza' });
10     res.sendStatus(204)
11   });
12 });
```

Proviamo quindi a chiamare questo endpoint con Postman e poi torniamo sul database: la pizza corrispondente è stata realmente e definitivamente eliminata!



# CRUD col DB: Show

Proviamo a leggere i dati di una singola pizza, sfruttando ancora l'id passato all'interno dell'endpoint:

```
router.get('/:id', (req, res) => {
  // recuperiamo l'id dall' URL
  const id = req.params.id

  const sql = 'SELECT * FROM pizzas WHERE id = ?';
  connection.query(sql, [id], (err, results) => {
    if (err) return res.status(500).json({ error: 'Database query failed' });
    if (results.length === 0) return res.status(404).json({ error: 'Pizza not found' });
    res.json(results[0]);
  });
});
```

# CRUD col DB: Show

Proviamo a leggere i dati di una singola pizza, sfruttando ancora l'id passato all'interno dell'endpoint:

```
router.get('/:id', (req, res) => {
  // recuperiamo l'id dall' URL
  const id = req.params.id

  const sql = 'SELECT * FROM pizzas WHERE id = ?';
  connection.query(sql, [id], (err, results) => {
    if (err) return res.status(500).json({ error: 'Database query failed' });
    if (results.length === 0) return res.status(404).json({ error: 'Pizza not found' });
    res.json(results[0]);
  });
});
```

```
{
  id: 1,
  name: 'Regina Margherita',
  image: 'img/pizzas/regina_margherita.jpg'
}
```

Benissimo, abbiamo i dettagli di una singola pizza.
Ma mancano le informazioni relative agli ingredienti.. come mai?



# CRUD col DB: Show

Proviamo a leggere i dati di una singola pizza, sfruttando ancora l'id passato all'interno dell'endpoint:

```
router.get('/:id', (req, res) => {
  // recuperiamo l'id dall' URL
  const id = req.params.id

  const sql = 'SELECT * FROM pizzas WHERE id = ?';
  connection.query(sql, [id], (err, results) => {
    if (err) return res.status(500).json({ error: 'Database query failed' });
    if (results.length === 0) return res.status(404).json({ error: 'Pizza not found' });
    res.json(results[0]);
  });
});
```

Ma certo! Le informazioni sugli ingredienti sono in un'altra tabella!



# CRUD col DB: Relazioni

Proviamo a modificare la funzione, in modo da ottenere anche le informazioni presenti nella tabella ingredienti. Per prima cosa, prepariamo 2 query:

1) La prima per recuperare la pizza giusta

2) La seconda, per recuperare gli ingredienti della pizza, aiutandoci con una JOIN

```
router.get('/:id', (req, res) => {
  // Recuperiamo l'id
  const { id } = req.params;

  // prepariamo la query per la pizza
  const pizzaSql = 'SELECT * FROM pizzas WHERE id = ?';

  // Prepariamo la query per gli ingredienti aiutandoci con una join e Where
  const ingredientsSql = `
    SELECT ingredients.*
    FROM ingredients
    JOIN ingredient_pizza
    ON ingredient_pizza.ingredient_id  = ingredients.id
    WHERE ingredient_pizza.pizza_id = ?
  `;

  // Continua...

});
```

# CRUD col DB: Relazioni

Ora dobbiamo eseguire entrambe le query, ma una dopo l'altra.

Partiamo quindi con la ricerca della pizza, e solo dopo averla individuata facciamo partire la seconda query, che ci restituirà gli ingredienti associati. Infine, aggiungiamo al nostro oggetto pizza la proprietà ingredients:

```
router.get('/:id', (req, res) => {
  /// Query preparate (vedi slide precedente)...

  // Eseguiamo la prima query per la pizza
  connection.query(pizzaSql, [id], (err, pizzaResults) => {
    if (err) return res.status(500).json({ error: 'Database query failed' });
    if (pizzaResults.length === 0) return res.status(404).json({ error: 'Pizza not found' });

    // Recuperiamo la pizza
    const pizza = pizzaResults[0];

    // Se è andata bene, eseguiamo la seconda query per gli ingredienti
    connection.query(ingredientsSql, [id], (err, ingredientsResults) => {
      if (err) return res.status(500).json({ error: 'Database query failed' });

      // Aggoiungiamo gli ingredienti alla pizza
      pizza.ingredients = ingredientsResults;
      res.json(pizza);
    });
  });
});
```



# LIVE CODING



# INSERT e UPDATE

Aggiungere e modificare dati

# CRUD col DB: Inserimenti e modifiche

I comandi SQL **INSERT** e **UPDATE** ci permettono rispettivamente di  effettuare inserimenti e modifiche nel nostro database. Vediamo come funzioano:

**INSERT**

**INTO** *`nome_tabella` (`colonna1`, `colonna2`)*

**VALUES** *('valore1', 'valore2')*

```sql
INSERT
INTO `pizzas`  (name, image)
VALUES ('Capricciosa', 'capricciosa.jpg')
```

**UPDATE** *`nome_tabella`*

**SET** *`colonna1` = 'valore1', `colonna2` = 'valore2'*

**WHERE** `id` = *id*

```sql
UPDATE `pizzas`
SET `name` = 'Americana', `image` = 'americana.jpg'
WHERE `id` = 1
```

*Le operazioni di update hanno sempre bisogno di un **WHERE** per identificare quale elemento modificare!*



# CRUD col DB: Store

Proviamo ora ad aggiungere una pizza al nostro DB. Per poterlo fare, abbiamo bisogno di creare una query SQL di inserimento:grazie al comando **INSERT**

```
1  router.post('/', (req, res) => {
2      //recuperiamo i dati dal corpo della richiesta
3    const { name, image } = req.body;
4    // prepariamo la query
5    const sql = 'INSERT INTO pizzas (name, image) VALUES (?, ?)'
6
7    // eseguiamo la query
8    connection.query(
9      sql,
10     [name, image],
11     (err, results) => {
12       if (err) return res.status(500).json({ error: 'Failed to insert pizza' });
13       res.status(201); // status corretto
14       console.log(results)
15       res.json({ id: results.insertId }); // restituiamo l'id assegnato dal DB
16     }
17   );
18 });
```

Notiamo che non abbiamo bisogno di fornire un **id:** sarà il DB ad assegnarne uno automaticamente!

Questo id viene restituito automaticamente in **`results.insertId`**



# CRUD col DB: Update

Proviamo ora a modificare una pizza con un'update:

```
function update(req, res) {

  // recuperiamo l'id dall' URL
  const { id } = req.params;

  // recuperiamo i dati dal body della richiesta
  const { name, image } = req.body;

  // Prepariamo la query per aggiornare la pizza
  connection.query(
    'UPDATE pizzas SET name = ?, image = ? WHERE id = ?',
    [name, image, id],
    (err) => {
      if (err) return res.status(500).json({ error: 'Failed to update pizza' });
      res.json({ message: 'Pizza updated successfully' });
    }
  );
}
```

Proviamo quindi a chiamare questo endpoint con Postman e poi torniamo sul database: la pizza corrispondente è stata realmente e definitivamente modificata!



# CRUD col DB: Relazioni (2)

Ok, ma gli ingredienti?

Per poter inserire e modificare anche gli ingredienti, la nostre funzioni devono aspettarsi un array di id degli ingredienti collegati alla pizza.

Una volta recuperati, si potranno effettuare delle chiamate aggiuntive per aggiornare le tabelle relazionate.

Nel caso di pizze e ingredienti, al momento di aggiungere una pizza avremmo da registrare nella tabella `ingredient_pizza` una riga per ciascun ingrediente, dove la colonna `pizza_id` avrebbe sempre il valore dell'id della pizza su cui stiamo lavorando, mentre nella colonna `ingredient_id` avremmo il valore del singolo ingrediente associato!

| ingredient_id | pizza_id |
|---|---|
| 1 | 5 |
| 2 | 5 |
| 7 | 5 |
| 8 | 5 |
| 9 | 5 |

# ESERCITAZIONE